

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATICS

CSIS 1624 – Practical 4

DATE: 13 August 2026

DUE DATE: End of practical sessions

- Your program **must** run without errors.
 - Include a comment block at the very top of **all files** with:
 - Your name
 - Student number
 - Practical number
 - Today's date
 - Missing any of these will result in a mark deduction.
 - Code that is commented out will not be marked.
 - Use meaningful comments to separate sections of your code.
 - Read through this entire document before starting.
-

1. Using Visual Studio 2019

1.1. Open the C# Windows Forms project (.NET Framework) App named **RestaurantFeedbackForm** in the **ThawSpace** (also called T: drive) on the lab computer. You may refer to the materials under LU3 on how to create a Windows Forms(.NET Framework) App in Visual Studio.

1.2. Add a comment block at the top of your **Program.cs** file above the using directives.

The comment block should list the following information:

- Your name and Surname
- Your student number
- Practical 4
- Today's date

Marks are deducted for failing to include this comment block in every file.

Open the provided **Windows Form (.NET Framework)** application and name the main form **CfrmMain**. Develop a simple feedback form for a restaurant (**see Figure 1**). The program should allow the user to specify their personal details, date of visit, overall dining experience rating, the specific aspects to rate, and the dining option used. When the user submits the form, a **MessageBox** should be displayed as in **Figure 2**. Assume that you do not need to implement defensive programming.

Step 1: Form Design

Design a form as in **Figure 1**. Set the properties of the Form and form controls as outlined below. Ensure that your form is neat and that all controls are aligned and adhere to naming conventions. Marks will be deducted for an untidy form and for controls that are not named correctly.

- **Form**
 - o **Name:** CfrmMain
 - o **FormBorderStyle:** FixedSingle
 - o **MaximizeBox, MinimizeBox:** false
 - o **StartPosition:** CenterScreen
 - o **Text:** Restaurant Feedback
- Labels – Add six labels with the following property settings:
 - o **Name:** lblName
 - o **Name:** lblEmail
 - o **Name:** lblVisitDate
 - o **Name:** lblOverallExperience
 - o **Name:** lblAspectsToRate
 - o **Name:** lblDiningOption
 - o Set the Text property for each label as displayed in the form in **Figure 1**.
- TextBox – Add two textboxes with the following property settings:
 - o **Name:** txtName
 - o **Name:** txtEmail
- DateTimePicker
 - o **Name:** dtpVisitDate
- RadioButtons – Add four radio buttons with the following property settings:
 - o **Name:** radExcellent
 - o **Name:** radGood
 - o **Name:** radAverage
 - o **Name:** radPoor
 - o Set the Text property for each radio button as displayed in the form in **Figure 1**.
- CheckBoxes – Add three checkboxes with the following property settings:
 - o **Name:** chkFoodQuality

- o **Name:** chkService
- o **Name:** chkAmbience
- o Set the Text property for each check box as displayed in the form in **Figure 1**.
- Combo box:
 - o **Name:** cmbDiningOption
- Buttons – Add two buttons with the following property settings:
 - o **Name:** btnSubmit
 - o **Name:** btnClear
 - o Set the Text property for each button as displayed in the form in Figure 1

Step 2: Develop the back-end program.

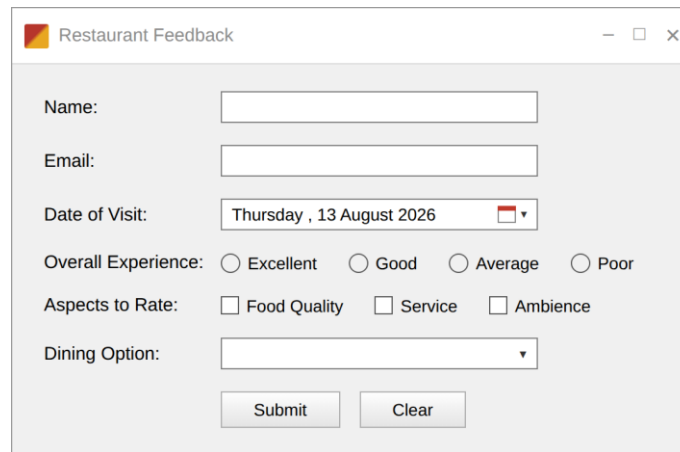
- In the form constructor, add items to the “Dining Option” drop-down list as in **Figure 3**. You do not need to set a default selected item.

Dining Options:

- **Dine-In**
- **Takeaway**
- **Delivery**
- **Private Event**
- **Buffet**
- **Business Lunch**
- **Catering**
- Create a method (**not** eventhandler) named GetSelectedRadioButton() to determine which radio button the user has selected. Use an appropriate if-else statement structure to determine which radio button has been selected. Return the value of the selected radio button's Text property.
- Develop one event handler that all three checkboxes will share. Name the event handler **AspectsToRateCheckedChanged** and assign it to the CheckedChanged event of each checkbox. Use if statements to determine if a checkbox has been selected. Append the value of the selected checkbox to a class-level variable named **sSelectedAspects**. Multiple checkboxes can be selected at once (see **Figure 2** and **Figure 4**). The checkbox options should be displayed as a comma-separated list (see **Figure 2**).
- Develop an event handler for the btnClear **Click** event. The event handler should either clear or reset the Form controls (see **Figure 4**). *Hint:* To reset the drop-down list, set the SelectedIndex property to -1.
- Develop an event handler for the btnSubmit **Click** event to display a summary of the feedback given. See **Figure 2** for the effect of this eventhandler. Note that the last checkbox option listed in the MessageBox has no comma at the end (*Hint:* use a method available to strings to **trim(TrimEnd)** the comma at the end). Take note the caption, MessageBox Button and MessageBox Icon displayed.

Bonus challenge (optional):

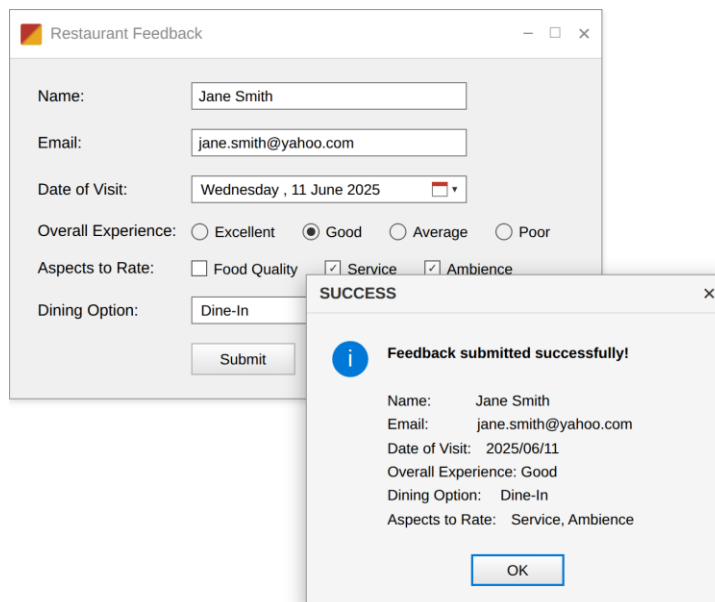
Adapt the code in the btnSubmit_Click eventhandler to use a compound condition to check if either textbox is empty (see **Figure 5**) and whether a drop-down list item has been selected (see **Figure 6** and **Figure 7**). If any of the afore-mentioned controls are empty, display a message (see **Figure 7**) Note the caption, MessageBox Button and MessageBox Icon displayed. Otherwise, if the fields have been adequately completed, display a MessageBox as in **Figure 2**.



The initial screen of the 'Restaurant Feedback' application. It features a form with the following fields and controls:

- Name: Textbox
- Email: Textbox
- Date of Visit: Date picker showing 'Thursday, 13 August 2026'
- Overall Experience: Radio buttons for 'Excellent', 'Good', 'Average', and 'Poor'.
- Aspects to Rate: Checkboxes for 'Food Quality', 'Service', and 'Ambience'.
- Dining Option: Dropdown menu.
- Submit and Clear buttons at the bottom.

Figure 1: Initial screen



The 'Restaurant Feedback' form is shown with the following data entered:

- Name: Jane Smith
- Email: jane.smith@yahoo.com
- Date of Visit: Wednesday, 11 June 2025
- Overall Experience: Good (selected)
- Aspects to Rate: Service and Ambience (checked)
- Dining Option: Dine-In

A 'SUCCESS' dialog box is displayed in the foreground, titled 'Feedback submitted successfully!'. It contains a summary of the submitted feedback:

- Name: Jane Smith
- Email: jane.smith@yahoo.com
- Date of Visit: 2025/06/11
- Overall Experience: Good
- Dining Option: Dine-In
- Aspects to Rate: Service, Ambience

The dialog box has an 'OK' button at the bottom.

Figure 2: The user has successfully submitted their feedback. A summary of the feedback is displayed.

The screenshot shows a web form titled "Restaurant Feedback". It contains the following fields and options:

- Name:** A text input field.
- Email:** A text input field.
- Date of Visit:** A date picker showing "Thursday , 13 August 2026".
- Overall Experience:** Four radio buttons labeled "Excellent", "Good", "Average", and "Poor".
- Aspects to Rate:** Three checkboxes labeled "Food Quality", "Service", and "Ambience".
- Dining Option:** A dropdown menu that is currently open, showing a list of options: "Dine-In", "Takeaway", "Delivery", "Private Event", "Buffet", "Business Lunch", and "Catering". "Dine-In" is highlighted as the selected option.

Figure 3: Drop-down list items.

The figure consists of two screenshots of the "Restaurant Feedback" form, connected by a right-pointing arrow, illustrating the effect of the "Clear" button.

Left Screenshot (Before Clear):

- Name:** "Jane Smith"
- Email:** "jane.smith@yahoo.com"
- Date of Visit:** "Wednesday , 11 June 2025"
- Overall Experience:** "Good" is selected.
- Aspects to Rate:** "Food Quality" and "Service" are checked.
- Dining Option:** "Dine-In"
- Buttons:** "Submit" and "Clear" (highlighted with a blue border).

Right Screenshot (After Clear):

- Name:** (Empty)
- Email:** (Empty)
- Date of Visit:** "Thursday , 13 August 2026"
- Overall Experience:** "Excellent" is selected.
- Aspects to Rate:** (All unchecked)
- Dining Option:** (Empty)
- Buttons:** "Submit" and "Clear".

Figure 4: The user clicked the "Clear" button and all controls were either cleared or reset.

Bonus challenge (optional):

The image shows a 'Restaurant Feedback' form window. The form contains the following fields: 'Name' (empty text box), 'Email' (empty text box), 'Date of Visit' (dropdown menu showing 'Thursday, 13 August 2026'), 'Overall Experience' (radio buttons for 'Excellent', 'Good', 'Average', 'Poor'), 'Aspects to Rate' (checkboxes for 'Food Quality', 'Service', 'Ambience'), and 'Dining Option' (dropdown menu showing 'Buffet'). A modal dialog box titled 'EMPTY FIELDS' is overlaid on the form, displaying a yellow warning triangle icon and the text 'Please fill in all required fields.' with an 'OK' button.

Figure 5: Only the textboxes are empty. A warning message is displayed. User is allowed to return to the form after clicking “OK”.

The image shows the same 'Restaurant Feedback' form window. In this state, the 'Name' text box is filled with 'Thabo', while the 'Email' text box remains empty. The 'Date of Visit' dropdown menu is still set to 'Thursday, 13 August 2026'. The 'Overall Experience' radio buttons, 'Aspects to Rate' checkboxes, and 'Dining Option' dropdown menu are also present. The 'EMPTY FIELDS' warning dialog box is still displayed, indicating that the form is incomplete due to the empty 'Email' field.

Figure 6: One textbox and drop-down list are empty. A warning message is displayed. User is allowed to return to the form after clicking “OK”.

Restaurant Feedback

Name:

Email:

Date of Visit: Thursday , 13 August 2026

Overall Experience: ☐ Excellent ☐ Good ☐ Average ☐ Poor

Aspects to Rate: ☐ Food Quality ☐ Service ☐ Ambience

Dining Option:

EMPTY FIELDS

 Please fill in all required fields.

OK

Figure 7: Textboxes and drop-down list are empty. A warning message is displayed. User is allowed to return to the form after clicking “OK”.

Submission process

The following steps outline the process you must follow at the end of **each** practical and test when you are ready to submit your Visual Studio solution.

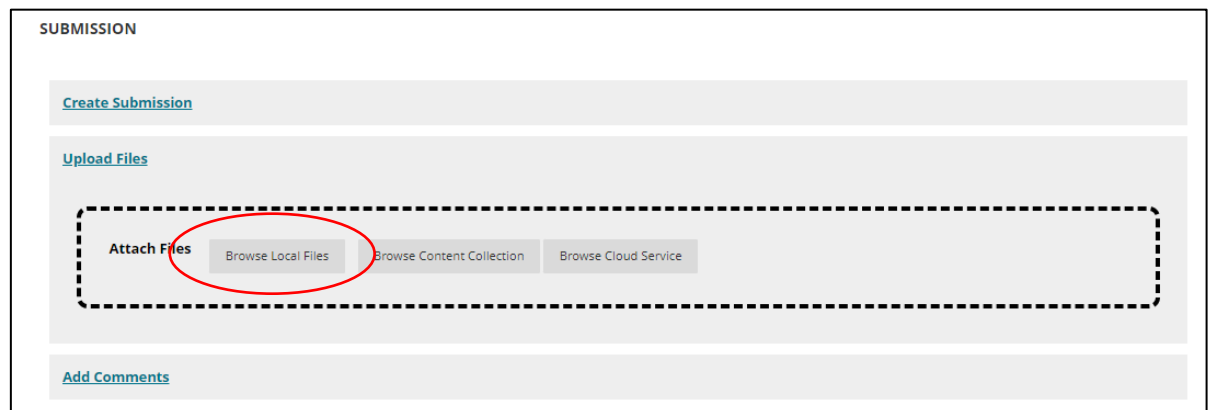
1. **Step 1: Save your project in a zipped file.**

To upload/share/send your project, you will need to zip the solution folder created by Visual Studio.

- 1.1. In the **File Explorer**, navigate/browse to where you have saved your Visual Studio solution folder in the **ThawSpace** (also called T: drive).
- 1.2. Right-click on the **solution folder** (the folder that contains **all** of your Visual Studio files including your .sln file).
- 1.3. In the menu that comes, navigate down to “**Send to**” and select “**Compressed (zipped) folder**”.
- 1.4. Name the zipped file as follows: **Surname (Student Number)** e.g., Doe (2023123456).
 - **Note:** Marks will be deducted in all practicals if your solution folder is not named correctly.
- 1.5. Press the “Enter” key on the keyboard.

2. **Step 2: Submit on Blackboard using the submission link.**

- 2.1. Navigate to the **CSIS1624** module on Blackboard.
- 2.2. In the left-hand menu, click on ***Practicals***.
- 2.3. Click on the link with the name of the current practical (in this case, “**Practical 4**”).
- 2.4. Under the section titled “**Submission**” in the page that pops up, click on “**Upload Files**” and the “**Browse Local Files**”.



The screenshot shows the 'SUBMISSION' page on Blackboard. It features a 'Create Submission' link at the top, followed by an 'Upload Files' section. Within this section, there is a dashed box containing three buttons: 'Attach Files', 'Browse Local Files', and 'Browse Content Collection'. The 'Browse Local Files' button is circled in red. Below this section is an 'Add Comments' link.

- 2.5. Use the **File Explorer** to navigate to your zipped file. Select the file and click the “Open” button.
- 2.6. Your file should be attached to Blackboard as below.

The screenshot shows the 'SUBMISSION' page in Blackboard. At the top, there is a 'Create Submission' link. Below it is the 'Upload Files' section, which contains a dashed box around the 'Attach Files' area. This area has three buttons: 'Browse Local Files' (which is highlighted with a blue border), 'Browse Content Collection', and 'Browse Cloud Service'. Below these buttons is a table titled 'Attached files' with columns 'File Name' and 'Link Title'. The table shows one file: 'Doe(2024123456).zip' with a 'Link Title' of 'Doe(2024123456).zip' and a 'Do not attach' link. At the bottom of the page is an 'Add Comments' link.

- 2.7. Click on the black “**Submit**” button at the bottom of the screen.
- 2.8. The following page will be displayed if your file has been uploaded successfully.

The screenshot shows the 'Review Submission History' page in Blackboard. At the top, there is a purple banner with a confirmation message: 'Your submission was successful! Your confirmation number is b484181cfbe34b20bc916dc19b4e956f. Copy and save this for your records. View all your submission receipts in My Grades.' Below the banner, the page title is 'Review Submission History: Practice Submission'. The main content area shows a large file icon and the filename 'Doe(2024123456).zip' with a 'Download' button. On the right side, there is a sidebar with 'Assignment Details' and 'GRADE' information. The 'GRADE' section shows 'LAST GRADED ATTEMPT' with a grade of '/1'. The 'ATTEMPT' section shows '2/21/24 7:56 AM' with a grade of '/1'. Below this, there is a 'Submission' section with a link to 'Doe(2024123456).zip' and a 'Download' button. At the bottom of the sidebar, there are 'OK' and 'Start New' buttons.

3. **Step 3: Check your submitted file.**

- 3.1. Download your newly submitted file and save it.
- 3.2. Right-click on the zipped file and select “Extract All”. Using the File Explorer, browse to where you would like to save your extracted folder.
- 3.3. Open your Visual Studio **.sln** file and make sure that you have submitted the correct folder.